Source: https://cam.feedzai.com/ds-studio-ds-api/25.0/reference/models/xgboost/

xgboost Â¶

XGBoostConfig Â¶

```
XGBoostConfig(reg_alpha: float = 0.0, max_bins: int = 256, dmatrix_type: str = 'auto',
skip_drop: float = 0.0, min_child_weight: float = 1.0, tree_method: str = 'auto', booster: str =
'gbtree', col_sample_rate: float = 1.0, max_delta_step: float = 0.0, max_leaves: int = 0, reg_la
mbda: float = 1.0, quiet_mode: bool = True, backend: str = 'auto', nthread: int | None = None, m
in_data_in_leaf: float = 0.0, rate_drop: float = 0.0, min_rows: float = 1.0, col_sample_rate_per_
tree: float = 1.0, ntrees: int = 50, gpu_id: int = 0, learn_rate: float = 0.3, colsample_bytree:
float = 1.0, min_split_improvement: float = 0.0, sample_rate: float = 1.0, eta: float = 0.3, sub_
sample: float = 1.0, max_abs_leafnode_pred: float = 0.0, gamma: float = 0.0, max_depth: int = 6,
colsample_bylevel: float = 1.0, score_tree_interval: int = 0, normalize_type: str = 'tree', min_
sum_hessian_in_leaf: float = 100.0, grow_policy: str = 'depthwise', one_drop: bool = False, samp
le_type: str = 'uniform', build_tree_one_node: bool = False, undersampling: float | None = None,
oversampling: float | None = None, seed: int | None = None, ignored_fields: set[str] | None = No
ne, schema: Schema | None = None)
```

Bases: `TrainingConfig`

The H2O XGBoost configuration object used for model training.

Parameters:

Name	Type	Description	Default
`reg_alpha` Â¶	`float`	L1 regularization.	`0.0`
`max_bins` Â¶	`int`	For tree_method=hist only: maximum number of bins.	`256`
`dmatrix_type` Â¶	`str`	Type of DMatrix. For sparse, NAs and 0 are treated equally.	`'auto'`
`skip_drop` Â¶	`float`	For booster=dart only: skip_drop (0..1).	`0.0`
`min_child_weight` Â¶	`float`	(same as min_rows) Fewest allowed (weighted) observations in a leaf.	`1.0`
`tree_method` Â¶	`str`	Tree method.	`'auto'`
`booster` Â¶	`str`	Booster type.	`'gbtree'`
	`float`	(same as colsample_bylevel) Column sample rate (from 0.0 to 1.0).	`1.0`

Name	Type	Description	Default
`col_sample_rate` Â¶			
`max_delta_step` Â¶	`float`	(same as max_abs_leafnode_pred) Maximum absolute value of a leaf node prediction.	`0.0`
`max_leaves` Â¶	`int`	For tree_method=hist only: maximum number of leaves.	`0`
`reg_lambda` Â¶	`float`	L2 regularization.	`1.0`
`quiet_mode` Â¶	`bool`	Enable quiet mode.	`True`
`backend` Â¶	`str`	Backend. By default (auto), a GPU is used if available..	`'auto'`
`nthread` Â¶	`int | None`	Number of parallel threads that can be used to run XGBoost. Cannot exceed H2O cluster limits (-nthreads parameter). Defaults to maximum available.	`None`
`min_data_in_leaf` Â¶	`float`	For tree_method=hist only: the minimum data in a leaf to keep splitting.	`0.0`
`rate_drop` Â¶	`float`	For booster=dart only: rate_drop (0..1).	`0.0`
`min_rows` Â¶	`float`	(same as min_child_weight) Fewest allowed (weighted) observations in a leaf.	`1.0`
`col_sample_rate_per_tree` Â¶	`float`	(same as colsample_bytree) Column sample rate per tree (from 0.0 to 1.0).	`1.0`
`ntrees` Â¶	`int`	(same as n_estimators) Number of trees.	`50`
`gpu_id` Â¶	`int`	Which GPU to use.	`0`
`learn_rate` Â¶	`float`	(same as eta) Learning rate (from 0.0 to 1.0).	`0.3`
`colsample_bytree` Â¶	`float`	(same as col_sample_rate_per_tree) Column sample rate per tree (from 0.0 to 1.0).	`1.0`

Name	Type	Description	Default
`min_split_improvement` Â¶	`float`	(same as gamma) Minimum relative improvement in squared error reduction for a split to happen.	`0.0`
`sample_rate` Â¶	`float`	(same as subsample) Row sample rate per tree (from 0.0 to 1.0).	`1.0`
`eta` Â¶	`float`	(same as learn_rate) Learning rate (from 0.0 to 1.0).	`0.3`
`sub_sample` Â¶	`float`	(same as sample_rate) Row sample rate per tree (from 0.0 to 1.0).	`1.0`
`max_abs_leafnode_pred` Â¶	`float`	(same as max_delta_step) Maximum absolute value of a leaf node prediction.	`0.0`
`gamma` Â¶	`float`	(same as min_split_improvement) Minimum relative improvement in squared error reduction for a split to happen.	`0.0`
`max_depth` Â¶	`int`	Maximum tree depth.	`6`
`colsample_bylevel` Â¶	`float`	(same as col_sample_rate) Column sample rate (from 0.0 to 1.0).	`1.0`
`score_tree_interval` Â¶	`int`	Score the model after every so many trees. Disabled if set to 0.	`0`
`normalize_type` Â¶	`str`	For booster=dart only: normalize_type.	`'tree'`
`min_sum_hessian_in_leaf` Â¶	`float`	For tree_method=hist only: the minimum sum of hessian in a leaf to keep splitting.	`100.0`
`grow_policy` Â¶	`str`	Grow policy - depthwise is standard GBM, lossguide is LightGBM.	`'depthwise'`
`one_drop` Â¶	`bool`	For booster=dart only: one_drop.	`False`
`sample_type` Â¶	`str`	For booster=dart only: sample_type.	`'uniform'`
	`bool`		`False`

Name	Type	Description	Default
`build_tree_one_node` Â¶		Run on one node only.	
`undersampling` Â¶	`float`	The value for undersampling. Only this or oversampling can be used.	`None`
`oversampling` Â¶	`float`	The value for oversampling. Only this or undersampling can be used.	`None`
`seed` Â¶	`int`	The seed value.	`None`
`ignored_fields` Â¶	`set of str`	Set of names corresponding to the fields to be ignored when training a model.	`None`
`schema` Â¶	`Schema`	Pass a schema to enable conversion between descs and internal names. By not passing a schema, ds-api will assume field names are references to internal Pulse names, not the ones in the UI.	`None`

Examples:

```
>>> model_config = XGBoostConfig()
>>> model = model_project.train(
...     "model_name",
...     datasource,
...     "target_field",
...     model_config
... )
>>> model.waitUntilModelTrained()
>>> config_from_model = model.get_config()
```

backend `property` `writable` Â¶

```
backend
```

booster `property` `writable` Â¶

```
booster
```

build_tree_one_node `property` `writable` Â¶

```
build_tree_one_node
```

col_sample_rate `property` `writable` Â¶

```
col_sample_rate
```

col_sample_rate_per_tree `property` `writable` Â¶

```
col_sample_rate_per_tree
```

colsample_bylevel `property` `writable` Â¶

```
colsample_bylevel
```

colsample_bytree `property` `writable` Â¶

```
colsample_bytree
```

dmatrix_type `property` `writable` Â¶

```
dmatrix_type
```

eta `property` `writable` Â¶

```
eta
```

gamma `property` `writable` Â¶

```
gamma
```

gpu_id `property` `writable` Â¶

```
gpu_id
```

grow_policy `property` `writable` Â¶

```
grow_policy
```

ignored_fields `property` `writable` Â¶

```
ignored_fields
```

learn_rate `property` `writable` Â¶

```
learn_rate
```

max_abs_leafnode_pred `property` `writable` Â¶

```
max_abs_leafnode_pred
```

max_bins `property` `writable` Â¶

```
max_bins
```

max_delta_step `property` `writable` Â¶

```
max_delta_step
```

max_depth `property` `writable` Â¶

```
max_depth
```

max_leaves `property` `writable` Â¶

```
max_leaves
```

min_child_weight `property` `writable` Â¶

```
min_child_weight
```

min_data_in_leaf `property` `writable` Â¶

`min_data_in_leaf`

min_rows `property` `writable` Â¶

`min_rows`

min_split_improvement `property` `writable` Â¶

`min_split_improvement`

min_sum_hessian_in_leaf `property` `writable` Â¶

`min_sum_hessian_in_leaf`

normalize_type `property` `writable` Â¶

`normalize_type`

nthread `property` `writable` Â¶

`nthread`

ntrees `property` `writable` Â¶

`ntrees`

one_drop `property` `writable` Â¶

`one_drop`

oversampling `property` `writable` Â¶

`oversampling`

quiet_mode `property` `writable` Â¶

`quiet_mode`

rate_drop `property` `writable` Â¶

`rate_drop`

reg_alpha `property` `writable` Â¶

`reg_alpha`

reg_lambda `property` `writable` Â¶

`reg_lambda`

sample_rate `property` `writable` Â¶

`sample_rate`

sample_type `property` `writable` Â¶

`sample_type`

score_tree_interval `property` `writable` Â¶

```
score_tree_interval
```

seed `property` `writable` Â¶

```
seed
```

skip_drop `property` `writable` Â¶

```
skip_drop
```

sub_sample `property` `writable` Â¶

```
sub_sample
```

subclasses `class-attribute` `instance-attribute` Â¶

```
subclasses: List[JavaWrapper] = []
```

tree_method `property` `writable` Â¶

```
tree_method
```

undersampling `property` `writable` Â¶

```
undersampling
```

auto_cast `staticmethod` Â¶

```
auto_cast(config: JavaWrapper) -> XGBoostConfig
```

Creates a XGBoostConfig from the java object.

Parameters:

Name	Type	Description	Default
`config` Â¶	`JavaWrapper`	Java configuration object.	*required*

Returns:

Type	Description
`XGBoostConfig`	Configuration object according to its python representation.